Class06

Tanieng Huang

R Functions Lab

Q1: Write a function add() to add some numbers together. Make sure to add comment(s) to the function explaining why you do things.

a. Your first version of the function should add two input numbers together; e.g. add(4, 7) should return 11. [1 pt]

```
#version1
add_1 <- function(x,y){
  x+y
}
#adding first and second argument together
```

```
add_1(4,7)
```

```
[1] 11
```

b. Your second version of the function should add any numbers together that the user inputs; e.g. add(1, 2, 3, -4) should return 2. [1 pt]

```
add_2 <- function(x, ...) {
  #... = the function can take any number of arguments
  #and returns their total sum
  # using sum for total values
  sum(x,...)
}
```

```
#second version test:
add_2(1,2,3,-4)
```

```
[1] 2
```

Q2: Write a function generate_dna() that generates a random DNA sequence of a length input by the user; e.g. generate_dna(11) may return "ATTTAAGGCTG". Make sure to add comments to the function. Play with functions sample() and paste() in your R-Studio Console to see how they can help here. [1 pt]

```
generate_dna <- function(x) {
  # Generate a random DNA sequence with a specified length

 # of a specified length using the four DNA bases
  bases <- c("A", "T", "G", "C")

  dna_seq <- sample(bases, size = x, replace = TRUE)
  #using sample to pull all random nucleotides
  #size = length means choose as many characters as requested
  #replace = TRUE: bases allowed to repeat

  paste(dna_seq, collapse = "")
  #generate a string from dna_seq
  #collapse = "": joint all bases without spaces
}

#test:
generate_dna(11)
```

```
[1] "CTTCAGTTGAT"
```

Q3: Write a function generate_protein() that generates a random peptide/protein sequence of a length input by the user; e.g. generate_protein(6) may return "WQRTAG". Make sure to use the single-letter code for all 20 amino acids and use no other letters (see list of amino acids at the bottom of the page); make sure that individual amino acids can appear more than once. Make sure to add comments to the function. [1 pt]

```
generate_protein <- function(x) {
  # This function generates a random protein sequence

   # of a specified length using the 20 amino acid single-letter codes
  amino_acids <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I",
                   "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V")
  #sample(): randomly selects amino acids
```

```
  protein_seq <- sample(amino_acids, size = x, replace = TRUE)
  paste(protein_seq, collapse = "")
}

#test:
generate_protein(11)
```

```
[1] "APNPISMWLMA"
```

Q4: Use your generate_protein() function to generate peptides of lengths 5, 7, 9, and 11 amino acids. Take advantage of the base R function sapply() for this. [1 pt]

```
lengths <- c(5, 7, 9, 11)
peptides <- sapply(lengths, generate_protein)
peptides
```

```
[1] "DGQYD"       "ETAPKLM"     "EPLHLNMFA"   "SIAPTPLQHWA"
```

```
#sapply() applies a function to each element of a vector
#for later question I save it as a vector for better reference
```

Q5: Take each of your peptides from Q4 and run a BLASTP search (https://blast.ncbi.nlm.nih.gov/Blast.cgi) selecting the Non-redundant protein sequences (nr) database and homo sapiens (taxid: 9606) organism. For each search, before clicking BLAST check the Show results in a new window tab next to the BLAST button to easily allow you to run the blast search for all four peptides at the same time in four different tabs. Once each search is done, for the top aligned protein for each peptide, list the percent identity (Per. Ident) times coverage (Query Cover) (for example, in the example below, 100% Query Cover times 85.71% Per. Ident = 100% x 85.71% = 1.00 x 0.8571 x 100% = 85.71%); make sure to type out your calculations in your Quarto document [note: if any of your values are below 30% you are probably miscalculating something]. Upload your numbers for each peptide (length (aa) and max identity (%)) into the shared class excel data sheet (see Data Sheet link in the Canvas assignment). Also, for each datapoint, add your initials in the "Your_Initials" column of the data sheet. Before adding your initials, scan through other used initials; if anyone has the same initials as you, modify your initials to make them unique (your initials will be used below to identify your specific data points in a ggplot graph). [1 pt

Peptide length 5 aa [YKRWL]:

Select seq gb|MFT1575958.1| immunoglobulin heavy chain junction region [Homo sapiens], MFT1575958.1

Query Cover = 100% Per. Ident = 100.00% Calculation: 1.00 × 1.00 × 100% = 100.00%

Peptide length 7 aa [VRQFNNH]:

Select seq gb|MCD08846.1| immunoglobulin light chain junction region [Homo sapiens], MCD08846.1

Query Cover = 100% Per. Ident = 71.00% Calculation: 1.00 × 0.71 × 100% = 71.00%

Peptide length 9 aa [NKNVRYCSQ]:

immunoglobulin heavy chain junction region [Homo sapiens], MOP31528.1

Query Cover = 78% Per. Ident =85.71% Calculation: 0.8571× 0.78 × 100% =66.85%

Peptide length 11 aa [ERAVNHTKSLR]:

Query Cover = 91% Per. Ident = 66.67% Calculation: 0.91 × 0.6667 × 100% = 60.67%

Q6: Once there is a good amount of data in the shared excel sheet, export the sheet as a CSV UTF-8 file (Export option under the File tab) and save the file to your class working folder (i.e. the folder in which you are currently generating a Quarto document). In R-Studio, upload the csv file into a data frame using the command peptide_df <- read.csv("your_filename"). Use the head() function to list the first 10 rows (note that the default for head() is 6) of the peptide_df dataframe as a test that everything looks good. [1 pt

```
#file downloaded to working folder
peptide_df <- read.csv("BIMM143_peptide(Sheet1).csv")
#shows the first 10 rows
head(peptide_df, 10)
```

```
   Length_aa Max_Identity_perc Your_Initials  X X.1 X.2 X.3 X.4 X.5 X.6 X.7 X.8
1          5            100.00           JLA NA  NA  NA  NA  NA  NA  NA  NA  NA
2          7             85.71           JLA NA  NA  NA  NA  NA  NA  NA  NA  NA
3          9             66.85           JLA NA  NA  NA  NA  NA  NA  NA  NA  NA
4         11             55.00           JLA NA  NA  NA  NA  NA  NA  NA  NA  NA
5          5             80.00           SKB NA  NA  NA  NA  NA  NA  NA  NA  NA
```

```
6          7             85.71           SKB NA  NA  NA  NA  NA  NA  NA  NA  NA
7          9             66.85           SKB NA  NA  NA  NA  NA  NA  NA  NA  NA
8         11             64.00           SKB NA  NA  NA  NA  NA  NA  NA  NA  NA
9          5            100.00           KET NA  NA  NA  NA  NA  NA  NA  NA  NA
10         7             85.71           KET NA  NA  NA  NA  NA  NA  NA  NA  NA
   X.9 X.10 X.11 X.12 X.13 X.14 X.15 X.16 X.17 X.18
1   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
2   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
3   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
4   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
5   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
6   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
7   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
8   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
9   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
10  NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
```

Q7: Use ggplot2 to generate a box plot with Length_aa on the x-axis and Max_Identity_perc on the y-axis. Since the box plot will require categorical values for Length_aa, you may have to specify the x-axis as as.factor(Length_aa). Feel free to pretty up the plot by adding your own x/y-axis labels, graph title, etc using the labs() function and/or using your favorite theme using a theme() function (see the data-visualization_ggplot.pdf cheat sheet in Canvas for options). Optional: you will reuse this plot a few times below, so your subsequent coding will be simpler if you assign your box plot to an object such as 'p' (p <- ggplot(peptide_df) +….etc). [1 pt

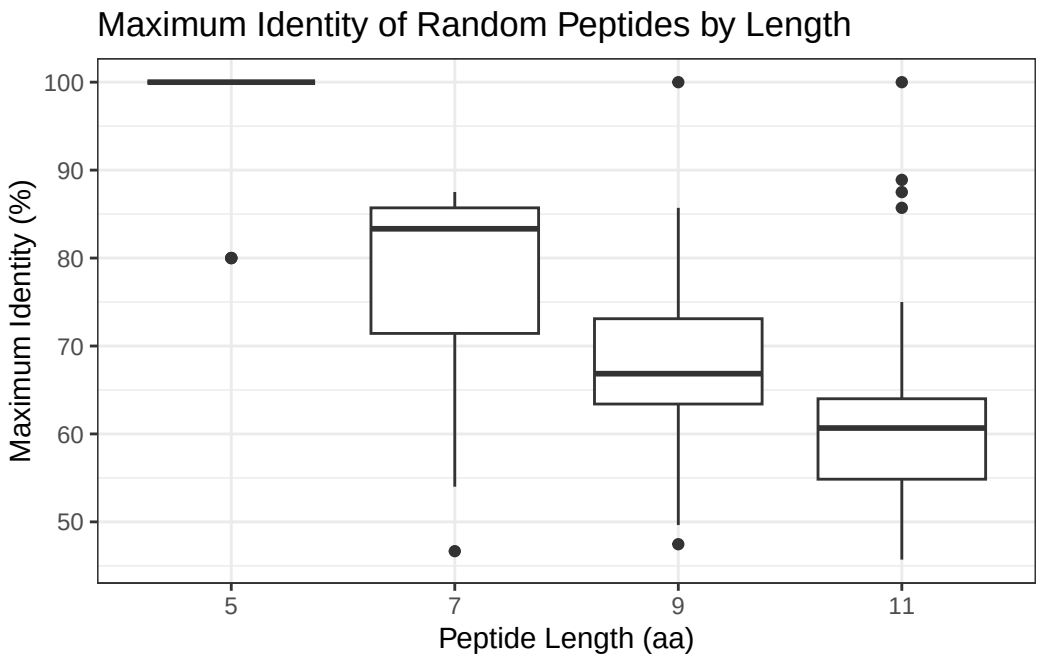
```
#blanks problem exist in file to fix with dplyr
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
peptide_df <- peptide_df %>%
  # Remove rows where Max_Identity_perc is missing
  filter(!is.na(Max_Identity_perc))
```

try the graph:

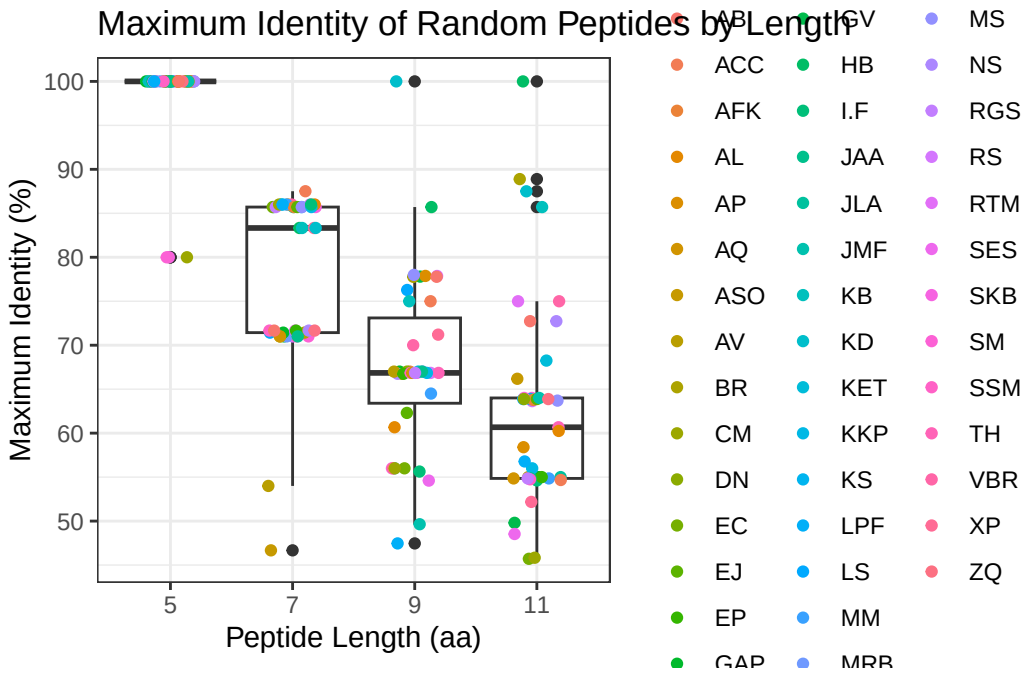
```r
library(ggplot2)

p <- ggplot(peptide_df, aes(x = as.factor(Length_aa), y = Max_Identity_perc)) +
  geom_boxplot() +
  ## Set the y-axis to a continuous numeric scale
# and show tick marks/labels from 0 to 100 in steps of 10
  scale_y_continuous(breaks = seq(0, 100, by = 10)) +
  labs(
    title = "Maximum Identity of Random Peptides by Length",
    x = "Peptide Length (aa)",
    y = "Maximum Identity (%)"
  ) +
  theme_bw()

p
```



Q8: Let's add a geom_jitter() layer to your box plot to visualize the individual data points on top of the box plot and highlight your own data points.

a. First, add a geom_jitter() layer to your box plot with each data point colored according to students' initials by adding the following additional line to your plot: + geom_jitter(aes(col=Your_Initials)). Feel free to narrow the spread of the jitter points using the width argument (e.g. width = 0.2). You can find your own datapoints this way via the color legend, but it looks rather messy. [1 pt

```
p + geom_jitter(aes(col = Your_Initials), width = 0.2)
```




```
#geom_jitter() adds each individual data point on top of the boxplot.
```

8b. Now let's control the coloring ourselves to better highlight your own data points. We can do this by generating a vector with a color for each datapoint (we will use your favorite color for your datapoints and "GREY" for all other datapoints). You can then use this vector to assign colors with the geom_jitter() call.

8b.i) First, use the rep() function to generate a vector consisting of as many repeats of "GREY" as there are datapoints in the peptide_df data frame. Assign this vector to an object you name color: i.e. color <- c(rep(,)) (fill in the blanks). Once you have generated the color object, use the table() function to ensure that you have the correct number of instances of "GREY" in color. [1 pt]

```
color <- c(rep("GREY", dim(peptide_df)[1]))
table(color)
```

```
color
GREY
 172
```

```
#dim(peptide_df) gives number of rows and columns
#dim(peptide_df)[1] extracts the number of rows
#rep("GREY", n) repeats "GREY" n times
#table(color) checks the count
```

8b.ii, Next, use the Your_Initials column in the peptide_df dataframe (which can be called as peptide_df$Your_Initials) to generate a TRUE/FALSE vector indicating which datapoints are yours (TRUE) and which are not (FALSE). Assign this vector to an object you name mydata: i.e. mydata <- _____ (fill in the blank). Again, check that you have the correct number of TRUE and FALSE in mydata using the table() function. [1 pt]

```
mydata <- peptide_df$Your_Initials == "TH"
table(mydata)
```

```
mydata
FALSE  TRUE
  168     4
```

```
#produces a logical vector: FALSE = others data
```

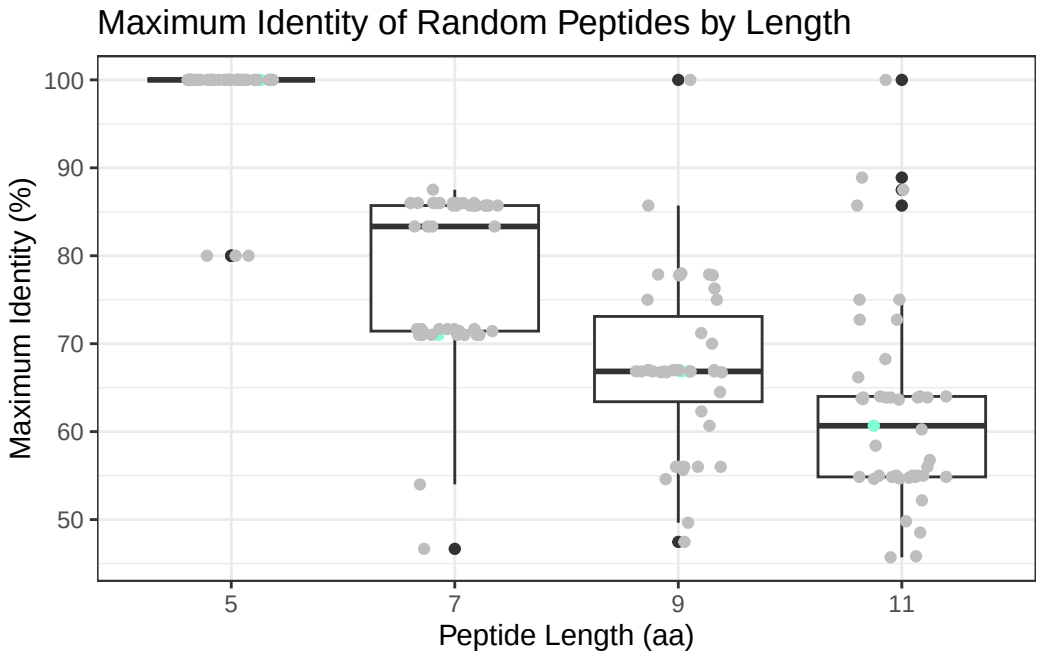
8b.iii) Now, use the mydata vector to replace the "GREY" values in the color vector that correspond to your data points with your favorite color, like this: color[mydata] <- "COLOR" (replace "COLOR" with the name of your favorite color; see link at the bottom of the page to a webpage with 657 R color names to select a cool color). Note, this will replace "GREY" with your favorite color for all the positions that are TRUE in mydata, i.e. the positions of your data points. Again, check the vector with table(). [1

```
color[mydata] <- "aquamarine"
table(color)
```

```
color
aquamarine       GREY
         4        168
```

8b.iv) Now you are ready to remake the plot by adding the following line to your original box plot from Q7: + geom_jitter(col=color). (Note that the color call is not run as an aesthetic here because we are now directly telling ggplot what color to use for each dot). Again, feel free to use the width argument to narrow the jitter points if you wish (e.g. width = 0.2). [1 pt]

```
#adding color to plot
p + geom_jitter(col = color, width = 0.2)
```



8c) Based on your boxplot, does any of your own peptide data fall outside the 25 to 75 percentile of the class's data? If yes, list which one(s). [1 pt]

Yes. My 7 aa data point falls slightly outside the 25th to 75th percentile range of the class data. My 7 aa maximum identity was 71.00%, while the lower boundary of the class interquartile range was about 72%. In contrast, my other values, 5 aa = 100.00%, 9 aa = 66.85%, and 11 aa = 60.67%, fall within the interquartile ranges for their respective groups.

Q9. Why might MHC class II peptide be 9 amino acids long instead of 5?

MHC peptides are likely around 9 amino acids long because a 9 aa sequence provides more sequence specificity than a 5 aa sequence. In our class data, 5 aa peptides showed extremely high similarity to human proteins, with the median at 100% and the interquartile range essentially also at 100%, meaning many 5 aa sequences can match human proteins almost perfectly by chance. In contrast, 9 aa peptides had a much lower median maximum identity of about 66%, with an interquartile range of about 63 to 73. This indicates that longer peptides are less likely to match unrelated self proteins by random chance. Therefore, peptides around 9 amino acids long would be better immune targets because they are more distinctive and more informative for discriminating self from non-self.